

Entrega de Engenharia de Software

Apresenta os requisitos da disciplina, como o projeto os atende e o arquivo, a página ou o diagrama que comprova cada um.

Período	Responsável
2026	**Equipe do projeto**

Sumário

Os requisitos da disciplina e as evidências

01
Visão geral 3
O que a disciplina pede e o que foi entregue

02
Requisitos mínimos 4
Planejamento, requisitos, diagramas e responsáveis

03
Requisitos extras 6
Padrões de projeto e testes automatizados

04
Entregas adicionais 7
O que o projeto reúne nesta área

05
Repositórios da disciplina 8
Papel de cada repositório na entrega

01

O projeto reúne planejamento, requisitos, diagramas, padrões de projeto e testes

O que a disciplina pede e o que a equipe entregou?

O que a disciplina pede

Engenharia de Software pede a organização do trabalho como projeto. O mínimo para a nota 7,0 tem cinco requisitos, o acompanhamento do planejamento em uma ferramenta como o Jira, o documento de requisitos funcionais, o diagrama de casos de uso, dois diagramas de atividades e a divisão de responsabilidades da equipe. Os extras pedem o uso de padrões de projeto e de um padrão de testes.

O que foi entregue

O planejamento é conduzido no Jira e documentado no Confluence, que também guarda o documento de engenharia de requisitos. O diagrama de casos de uso e os diagramas de atividades estão na galeria de diagramas do docs-warehouse, e as imagens em PNG ficam na pasta assets/diagrams, para quem não conseguir abrir a galeria. As áreas de atuação, os papéis do Scrum e a rotina de reuniões estão registrados, e o código usa padrões de projeto como Strategy, Transactional Outbox, Repository, Factory e Adapter, com testes automatizados executados pela integração contínua.

Como ler este documento

Cada requisito traz o identificador, o enunciado, a descrição do que foi entregue e um link direto para a evidência. A tabela resume os requisitos.

Item	Tipo	Requisito	Evidência de exemplo
M1	Mínimo	Acompanhamento do planejamento no Jira	Página Nosso Jira no Confluence
M2	Mínimo	Documento de requisitos funcionais	Página Engenharia de Requisitos no Confluence
M3	Mínimo	Diagrama de casos de uso	Galeria e PNG no docs-warehouse
M4	Mínimo	Dois diagramas de atividades	Galeria de diagramas do docs-warehouse
M5	Mínimo	Dinâmica de trabalho e responsáveis por área	Áreas de Atuação e Agenda Fixa no Confluence
E1	Extra	Padrões de projeto	strategies.py em api-recommendation
E2	Extra	TDD e testes unitários	src/test e TESTING.md

02

Os cinco requisitos do mínimo têm evidência no Confluence, nos diagramas e no GitHub

Como o projeto cumpre cada requisito do mínimo para 7,0?

M1. Criar um mecanismo de acompanhamento (planejamento) do projeto via Jira, Trello ou semelhante

A equipe planeja o projeto no Jira do ambiente interdesctruction.atlassian.net, nos projetos DEM, DEV, IA e DB, e a professora da disciplina foi adicionada a esse ambiente. O Confluence do mesmo ambiente descreve o uso da ferramenta nas páginas Nosso Jira, Criação de Issues e Story Points.

Links de exemplo, a página Nosso Jira no Confluence e o projeto DEV no Jira.

M2. Elaborar documento com os requisitos funcionais do sistema

A página Engenharia de Requisitos reúne os tipos de usuário e os requisitos funcionais e não funcionais.

Link de exemplo, Engenharia de Requisitos no Confluence.

M3. Elaborar o diagrama de casos de uso com todos os casos, relacionados aos requisitos funcionais

O diagrama de casos de uso está na galeria de diagramas do docs-warehouse, com cinco atores (visitante, profissional, membro da empresa, administrador da empresa e administrador da plataforma) e 16 casos de uso ligados a eles. O board Miro do projeto também o reúne. A imagem em PNG do diagrama fica na pasta assets/diagrams do docs-warehouse.

Links de exemplo, o diagrama de casos de uso na galeria do docs-warehouse e a imagem casos-de-uso.png em docs-warehouse.

M4. Escolher 2 processos do sistema e elaborar um diagrama de atividades para cada um

Os dois processos escolhidos estão desenhados como diagramas de atividades na galeria de diagramas do docs-warehouse, com início, ações, decisões, fim e raias por responsável. O primeiro é o cadastro do profissional, que passa pela validação de IA e pela revisão do administrador da plataforma. O segundo é a proposta e a execução de um projeto, entre demandante, fornecedor, plataforma e profissional. As imagens em PNG dos dois diagramas ficam na pasta assets/diagrams do docs-warehouse, para quem

não conseguir abrir a galeria. Os repositórios ai-assistant, databricks-stocker e database-console trazem ainda fluxos em Mermaid.

Links de exemplo, o diagrama Cadastro do profissional na galeria e a pasta assets/diagrams em docs-warehouse.

M5. Documento com a dinâmica de trabalho da equipe e o responsável por back, front, banco de dados etc.

A página Áreas de Atuação (2º Ano) divide o trabalho em back-end, front-end, banco de dados, BI, IA, infraestrutura, design e Scrum. Ela define o product owner, o scrum master e os revisores oficiais, e cada membro aparece com as suas áreas e com o revisor de outra área. Os repositórios api-core, api-auth, api-recommendation, web-app e infra-platform têm ainda um arquivo CODEOWNERS com o responsável pelo código.

O trabalho é acompanhado em quatro espaços do Jira. O DEM guarda as tarefas gerais, o DEV cobre back-end, front-end e a infraestrutura do GitHub, o IA cobre a inteligência artificial e o DB cobre BI e banco de dados. Uma tarefa passa pelos estados tarefas pendentes, em espera, em desenvolvimento, em análise, em testes e concluída, com o estado bloqueado quando há impedimento. A pull request fica em análise até ser aprovada.

A rotina tem uma weekly às quartas-feiras, no intervalo, em que cada área do primeiro ano conversa com a área correspondente do segundo ano. Um membro do segundo ano acompanha uma área diferente a cada weekly. Nos últimos 30 dias antes da entrega, o segundo ano faz reuniões diárias.

Links de exemplo, a página Áreas de Atuação (2º Ano) no Confluence e a página Agenda Fixa no Confluence.

03

O código usa cinco padrões de projeto e roda testes a cada PR

Como o projeto cumpre os requisitos extras de Engenharia de Software?

E1. Usar algum padrão de projeto, indicar em que parte do código e com que finalidade

A tabela indica cada padrão, o arquivo em que ele aparece e a finalidade. O padrão Strategy se repete nos três feeds de recomendação, para ofertas, profissionais e fornecedores.

Padrão	Onde	Finalidade
Strategy	strategies.py dos feeds de ofertas, profissionais e fornecedores em api-recommendation	Regras de ranking intercambiáveis, como a estratégia best_match de profissionais
Transactional Outbox	OutboxPollingScheduler.kt em api-auth	Publicar os eventos de usuário na Redis Stream sem perder mensagem
Repository	Pasta repository em api-core, com Spring Data JPA	Isolar o acesso a dados do domínio
Factory	ProblemDetailFactory.java em api-core	Criar respostas de erro padronizadas (RFC 7807)
Adapter	RecommendationClient.java em api-core	Encapsular a chamada ao api-recommendation

Link de exemplo, strategies.py de ofertas em api-recommendation.

E2. Utilizar um padrão de testes e indicar onde será utilizado e como

A equipe utilizou TDD no desenvolvimento, e os repositórios têm testes unitários. O projeto mantém testes automatizados nos repositórios api-core, api-auth, api-messenger, api-notify, api-email, api-recommendation, api-google, os assistentes de IA, lib-ai, lib-web, web-app e mobile-app. O api-core tem testes unitários e de integração na pasta src/test, e o fluxo de integração contínua em ci.yml executa os testes com cobertura a cada pull request. O web-app tem arquivos de teste ao lado dos serviços, como analytics.service.test.ts, e o ai-assistant guarda os testes do agente na pasta tests.

Links de exemplo, a pasta src/test em api-core e o padrão de testes em TESTING.md em docs-warehouse.

04
Entregas adicionais

O que o projeto reúne nesta disciplina?

O projeto mantém decisões de arquitetura em ADRs no ai-assistant, fluxos em Mermaid mantidos junto ao código e o repositório docs-warehouse, que concentra os padrões de documentação, de erros, de OpenAPI e de saúde dos serviços usados por todos os repositórios. O generic-template oferece a cada novo repositório um README, uma coleção Bruno, a integração contínua e o modelo de pull request, e o repositório .github disponibiliza workflows reutilizáveis de release-please, publicação de imagem, promoção para o GitOps e verificação do título das pull requests.

05

Onze repositórios fazem parte da disciplina

Onde está o código e o documento de cada requisito?

Repositório	Papel na disciplina	Requisitos
ai-assistant	Fluxo em Mermaid de um turno de conversa e testes do agente	M4, E2
databricks-stocker	Fluxo em Mermaid do pipeline de dados	M4
database-console	Fluxo em Mermaid da carga de dados	M4
api-core	Padrões Repository, Factory e Adapter, testes e integração contínua	M5, E1, E2
api-auth	Padrão Transactional Outbox e responsável pelo código	M5, E1
api-recommendation	Padrão Strategy nos feeds de recomendação e responsável pelo código	M5, E1
web-app	Responsável pelo código e testes de serviços	M5, E2
infra-platform	Responsável pelo código	M5
docs-warehouse	Galeria de diagramas, imagens em PNG dos casos de uso e dos fluxos e padrões de documentação, de erros, de OpenAPI e de saúde dos serviços	M3, M4, Entregas adicionais
generic-template	Estrutura inicial com README, coleção Bruno, integração contínua e modelo de pull request para novos repositórios	Entregas adicionais
.github	Workflows reutilizáveis disponíveis aos repositórios, de release, publicação de imagem e verificação de pull requests	Entregas adicionais



Entrega acadêmica.